



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA
DIPARTIMENTO DI ECCELLENZA



CORSO DI LAUREA IN INFORMATICA

Tecnologie Software per il Web

Espressioni Regolari

PROF. ATTILIO DELLA GRECA

a.a. 2025-2026

Cosa sono le espressioni regolari?

Una **regular expression** (o *regex*) è un oggetto che **descrive un pattern di caratteri**.

È uno strumento potentissimo per **analizzare**, **validare** e **trasformare** stringhe di testo.



Pattern-matching

Cercare nel testo sequenze che rispettano un determinato schema (es. un indirizzo email, un numero di telefono, una data...)



Search-and-replace

Trovare tutte le occorrenze di un pattern e sostituirle con un'altra stringa (es. ripulire HTML, normalizzare spazi, censurare parole)

Sintassi: come si scrive una regex

/ **^** **[A-z]** **+** **\$** **/g**

delimitatore

pattern

modificatori



Sintassi nativa

In JavaScript le regex hanno una sintassi letterale: `/.../`, senza virgolette (come stringhe in Java).



Caratteri speciali

La maggior parte è in comune con Java, Unix e Perl (`^ $ . * + ? \ [...]`).



Costruzione alternativa

Esiste anche `new RegExp("pattern", "flags")`, utile quando il pattern è dinamico.

Ancore e caratteri speciali

^

Inizio della stringa

`/^abc/` → trova 'abc' SOLO se all'inizio

\$

Fine della stringa

`/abc$/` → trova 'abc' SOLO se alla fine

.

Qualunque carattere

`/h.t/` → 'hat', 'hit', 'h@t' ... (no newline)

\

Escape: tratta come letterale

`/3\.14/` → trova letteralmente '3.14'

Quantificatori

Quante volte si ripete il pattern

zero o più volte

`/ab*/` → `'a'`, `'ab'`,
`'abb'`, `'abbb'`...

+

una o più volte

`/ab+/` → `'ab'`, `'abb'`,
`'abbb'`...

?

zero o una volta

`/colou?r/` → `'color'` e
`'colour'`

{n}

esattamente n volte

`/\d{5}/` → esattamente 5
cifre (CAP)

{n,}

n o più volte

`/\d{2,}/` → almeno 2
cifre

{n,m}

da n a m volte

`/\d{2,4}/` → da 2 a 4
cifre

Classi di caratteri

Gruppi di caratteri ammessi (o vietati)

Gruppi personalizzati

[abc] uno tra a, b o c

[a-z] qualsiasi lettera minuscola

[A-Z] qualsiasi lettera maiuscola

[A-z] maiuscole o minuscole (sconsigliato!)

[0-9] una cifra qualunque

[^abc] **NON** uno tra a, b o c

Scorciatoie predefinite

\d una cifra (digit) → [0-9]

\D NON una cifra

\w un word char (lettera, cifra o _)

\W NON un word char

\s uno spazio (space, tab, newline)

\S NON uno spazio

Alternative, gruppi e lookahead

Costruire pattern complessi

(x|y)

Alternative Una delle alternative specificate.

/ (cat|dog) / → trova 'cat' oppure 'dog'

(?=n)

Lookahead positive Stringa SEGUITA da n (n non viene consumato nel match).

/ Jack(=Sprat) / → 'Jack' solo se seguito da 'Sprat'

(...)

Gruppi di cattura Raggruppa più caratteri per applicare quantificatori o estrarli.

/ (ab)+ / → 'ab', 'abab', 'ababab'...

(?!n)

Lookahead negative Stringa NON seguita da n.

/ Jack(?!Sprat) / → 'Jack' solo se NON seguito da 'Sprat'

Modificatori (flags)

Cambiano il comportamento del pattern



global

Trova TUTTE le corrispondenze, non solo la prima.

```
/x/g → applica su tutto il testo
```



case-insensitive

Ignora le differenze tra maiuscole e minuscole.

```
/abc/i → trova 'abc', 'ABC', 'AbC'...
```



multiline

^ e \$ funzionano per ogni riga, non solo per la stringa.

```
/^x/m → ogni inizio riga
```


Metodi delle stringhe che usano regex

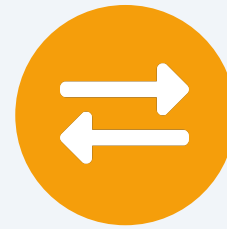
Le quattro operazioni fondamentali



`str.match(re)`

Ritorno:

Array con le parti che fanno match



`str.replace(re)`

Ritorno:

Nuova stringa con sostituzioni



`str.split(re)`

Ritorno:

Array delle porzioni tra i match



`str.search(re)`

Ritorno:

Indice della prima corrispondenza

Metodi in azione

Esempi con la stringa "Axbxxcxxx"

`"Axbxxcxxx".match(/x+/g)` → `["x", "xx", "xxx"]`

*Restituisce un array di parti della stringa che corrispondono all'espressione regolare (**tutte le sequenze di 'x'**)*

`"Axbxxcxxx".replace(/x+/g, "q")` → `"Aqbqcqd"`

*Sostituisce tutte le occorrenze che corrispondono con una stringa di sostituzione (**sostituisce ogni sequenza con 'q'**)*

`"Axbxxcxxx".split(/x+/)` → `["A", "b", "c", "d"]`

*Restituisce un array di tutte le parti della stringa che si trovano tra le espressioni regolari (**spezza la stringa sulle 'x'**)*

`"Axbxxcxxx".search(/x+/)` → `1`

*Restituisce la posizione del primo posto che corrisponde all'espressione regolare (**posizione della prima 'x'**)*

Approccio pratico

JavaScript code for validating user input

```
function validateInput(obj) {  
    var pattern = /.../;  
    if(obj.value.match(pattern)) {  
        //INPUT VALIDO Do something: set class  
        return true;  
    } else {  
        //INPUT NON VALIDO Do something: set error  
        class, focus, show error message,  
        show suggestion,  
        return false;  
    }  
}
```



Match → valido

Se la regex fa match l'input rispetta il formato richiesto.



No match → errore

Mostra un messaggio chiaro, applica una classe CSS di errore e dai focus al campo.



Riusa la funzione

Una stessa funzione può essere parametrizzata con pattern e messaggi diversi per ogni campo.

Esempio 1 — Username (solo lettere)

Validare un nome utente fatto di sole lettere

`/^[A-z]+$ /g`

```
function allLetter(uname) {  
  var letters = /^[A-z]+$ /g;  
  if (uname.value.match(letters)) {  
    return true;  
  } else {  
    alert("Solo caratteri alfabetici");  
    return false;  
  }  
}
```

Come si legge

`^` inizio stringa

`[A-z]` una lettera (Maiusc/Min)

`+` una o più volte

`$` fine stringa

`/g` modalità global

Esempio 2 — Indirizzo

Parole separate da spazi (es. via, città, civico)

`/^\\w+(\\s\\w+)+$/g`

```
function alphaNumSpaces(uadd) {  
  var letters = /^\\w+(\\s\\w+)+$/g;  
  if (uadd.value.match(letters)) {  
    return true;  
  } else {  
    alert("Parole separate da spazi");  
    return false;  
  }  
}
```

Come si legge

`\\w+` una parola

`(\\s\\w+)+` spazio + parola, almeno una volta

`^...$` l'intera stringa rispetta il pattern

"via giovanni paolo II" ✓
valido

Esempio 3 — mail

la regex completa per le email è molto complessa! Questa è una semplificazione:

```
/^\S+@\S+\.\S+$/g
```

```
function validateEmail(uemail) {  
  var mailformat = /^\S+@\S+\.\S+$/g;  
  if (uemail.value.match(mailformat)) {  
    return true;  
  } else {  
    alert("Email non valida");  
    return false;  
  }  
}
```

Significato

\S+ uno o più caratteri non-spazio

@ la chiocciola letterale

\. un punto letterale (escape!)

*Senza l'escape **\.** un **.** significa 'qualunque carattere'.*

Esempio 4a — Telefono di 10 cifre

Esattamente 10 cifre consecutive

`/^\d{10}$/`

```
function validateEmail(uemail) {  
    var phoneno = /^\d{10}$/;  
    if (inputtxt.value.match(phoneno)) {  
        return true;  
    } else {  
        alert("Numero non valido");  
        return false;  
    }  
}
```

Significato

`^` inizio stringa

`\d` una cifra [0-9]

`{}` esattamente n volte

`$` fine stringa

Esempio 4a — Telefono formattato

Formato (XXX)-XXX-XXXX → parentesi e trattini come letterali

```
/^\(\d{3}\)-\d{3}-\d{4}$/
```

```
function validateEmail(uemail) {  
  var phoneno = /^\(\d{3}\)-\d{3}-\d{4}$/;  
  if (inputtxt.value.match(phoneno)) {  
    return true;  
  } else {  
    alert("Numero non valido");  
    return false;  
  }  
}
```

Significato

- `\(` parentesi tonda APERTA letterale
- `\d{3}` esattamente 3 cifre
- `\)` parentesi tonda CHIUSA letterale
- `-` trattino letterale (no escape)
- `\d{4}` esattamente 4 cifre finali

Esempio 4a — Telefono formattato

Formato (XXX)-XXX-XXXX → parentesi e trattini come letterali

```
/^\(\d{3}\)-\d{3}-\d{4}$/
```

```
function validateEmail(uemail) {  
  var phoneno = /^\(\d{3}\)-\d{3}-\d{4}$/;  
  if (inputtxt.value.match(phoneno)) {  
    return true;  
  } else {  
    alert("Numero non valido");  
    return false;  
  }  
}
```

Esempio di test

"(123)-456-7890"

✓ formato valido

Esempi NON validi:

123-456-7890

(123) 456-7890

(12)-456-7890

match vs. test

Due modi per controllare le corrispondenze

`regex.test( string )`

Restituisce true o false.

Cerca un match e ti dice solo SE esiste, senza estrarre nulla. Perfetto quando ti serve un booleano.

```
/^\d+$/  
.test("123");   → true  
.test("12a");   → false
```

`string.match( regex )`

Restituisce un Array oppure null.

Estrae le corrispondenze. Siccome **null** viene valutato come **false**, può essere usato come booleano in un **if**.

```
"abc123".match(/\d+/);  
→ ["123"]
```

Esempio test — Data e ora

Pattern data-mese-anno ora:minuti, lunghezze variabili

```
/\d{1,2}-\d{1,2}-\d{4} \d{1,2}:\d{2}/
```

Input:

"30-5-2017 11:25"



`dateTime.test(...)` = **true**

Tutti i pezzi rispettano la regex.

Input:

"30-5-2017 11:5"



`dateTime.test(...)` = **false**

I minuti hanno solo 1 cifra, ma il pattern richiede esattamente 2 (`\d{2}`).

Riepilogo

Ancore

^

inizio

\$

fine

.

qualunque carattere

escape

Quantificatori

0 o più

+

1 o più

?

0 o 1

{n,m}

da n a m

Classi

\d

cifra

\w

word char

\s

spazio

[^x]

non x

Flags

g

global

i

case-insens.

m

multiline

Dynamic form and validation (DynamicForm.zip)

Registration

Information

Name:
Surname:
Email:

Phone:

Registration

Information

Name:
Surname:
Email:

Phone:
Phone:
Phone:

Registration

Information

Name:
Surname:
Email:

Phone:

The form

```

</head>
<body>
<h3>Registration</h3>
<form id="dForm" action="Registration" onsubmit="event.preventDefault(); validate(this)">
<fieldset>
  <legend>Information</legend>
  <label for="firstname">Name:</label>
  <input type="text" name="firstname" placeholder="name" required>
  <br>
  <label for="lastname">Surname:</label>
  <input type="text" name="lastname" placeholder="surname" required>
  <br>
  <label for="email">Email:</label>
  <input type="text" name="email" placeholder="mrisi@unisa.it" required>
  <br>
  <hr>
  <div id="phones">
    <label for="number">Phone:</label>
    <input class="backYellow" type="text" name="number" placeholder="111-1111111" required>
    <input type="button" value="+" onclick="addPhone()">
  </div>
  <br>
  <input type="submit" value="Register">&nbsp;
  <input type="reset" value="Reset">
</fieldset>
</form>

</body>
</html>

```

CSS

```
<style>
  body {
    width: 400px;
    margin: 0 auto;
  }
  legend {
    padding: 3px;
    border: 1px solid purple;
    border-radius: 3px;
  }
  fieldset {
    border: 1px solid purple;
    border-radius: 7px;
  }
  .backYellow {
    background-color: yellow;
  }
  .error {
    background-color: red;
  }
  hr {
    border: 0;
    border-top: 1px solid purple;
    -webkit-margin-start: -0.8em;
    -webkit-margin-end: -0.8em;
  }
</style>
```

Regular expressions

```
function checkNamesurname(inputtxt) {
    var name = /^[A-Za-z]+$/;
    if(inputtxt.value.match(name))
        return true;

    return false;
}

function checkEmail(inputtxt) {
    var email = /^[w+([\.-]?w+)*@w+([\.-]?w+)*(\.\w{2,3})+$/;
    if(inputtxt.value.match(email))
        return true;

    return false;
}

function checkPhonenumber(inputtxt) {
    var phoneno = /^( [0-9]{3}-[0-9]{7})$/;
    if(inputtxt.value.match(phoneno))
        return true;

    return false;
}
```


Validation

```
function validate(obj) {  
    var valid = true;  
  
    var name = document.getElementsByName("firstname")[0];  
    if(!checkNamesurname(name)) {  
        valid = false;  
        name.classList.add("error");  
    } else {  
        name.classList.remove("error");  
    }  
  
    var surname = document.getElementsByName("lastname")[0];  
    if(!checkNamesurname(surname)) {  
        valid = false;  
        surname.classList.add("error");  
    } else {  
        surname.classList.remove("error");  
    }  
  
    var email = document.getElementsByName("email")[0];  
    if(!checkEmail(email)) {  
        valid = false;  
        email.classList.add("error");  
    } else {  
        email.classList.remove("error");  
    }  
  
    var numbers = document.getElementsByName("number");  
    for(var i=0; i < numbers.length; i++) {  
        if(!checkPhonenumber(numbers[i])) {  
            valid = false;  
            numbers[i].classList.add("error");  
        } else {  
            numbers[i].classList.remove("error");  
        }  
    }  
  
    if(valid) obj.submit();  
}
```

Add/Remove phone input fields

```

var count = 2;

function addPhone() {
    var container = document.getElementById("phones");

    var divv = document.createElement("div");
    divv.id = "id"+count;
    count++;

    var label = document.createElement("label");
    label.htmlFor = "number";
    label.appendChild(document.createTextNode("Phone:"));
    divv.appendChild(label);

    var element = document.createElement("input");
    element.type = "text";
    element.name = "number";
    element.placeholder = "111-1111111";
    element.required = "required";
    element.className = "backYellow";
    divv.appendChild(element);

    var input = document.createElement("input");
    input.type = "button";
    input.value = "-";
    input.addEventListener("click", function() {removePhone(divv.id)});
    divv.appendChild(input);

    container.appendChild(divv);
}

function removePhone(idd) {
    var element = document.getElementById(idd);
    element.parentNode.removeChild(element);
}

```

Risorse

- JavaScript RegExp Reference
 - [w3schools.com/jsref/jsref_obj_regexp.asp](https://www.w3schools.com/jsref/jsref_obj_regexp.asp)
- Regex tester interattivo
 - regex101.com
- Form with Multiple Steps
 - https://www.w3schools.com/howto/howto_js_form_steps.asp
- Autocomplete
 - https://www.w3schools.com/howto/howto_js_autocomplete.asp
- Modal Login Form
 - https://www.w3schools.com/howto/howto_css_login_form.asp
- Checkout Form
 - https://www.w3schools.com/howto/howto_css_checkout_form.asp
- Form with Icons
 - https://www.w3schools.com/howto/howto_css_form_icon.asp
- Password Validation
 - https://www.w3schools.com/howto/howto_js_password_validation.asp